

SymPy Bug Report

1 Bug 6: Piecewise Left-Hand Limit Returns Right Branch

1.1 Minimal Reproducer

```
from sympy import Piecewise, Rational, limit, symbols

x = symbols("x", real=True)
p = Piecewise((0, x < 0), (1, True))

print(limit(p, x, 0, dir="-"))
print(p.subs(x, Rational(-1, 10)))
print(p.subs(x, Rational(1, 10)))
```

1.2 Observed Behavior

```
limit(Piecewise((0, x < 0), (1, True)), x, 0, dir='-') = 1
left sample p.subs(x, -1/10) = 0
right sample p.subs(x, 1/10) = 1
```

SymPy 1.14.0 returns 1, the value of the default branch, for the left-hand limit. The same expression evaluates to 0 at a nearby point on the left side, so the returned value is the value from the wrong side of the jump.

1.3 Expected Behavior

The left-hand limit should be 0:

$$\lim_{x \rightarrow 0^-} \text{Piecewise}((0, x < 0), (1, \text{True})) = 0.$$

Software-wise, `limit(p, x, 0, dir="-")` should select the branch that is active in a punctured left neighborhood of 0, not the branch that is active at or to the right of the boundary.

1.4 Mathematical Explanation

For every real x with $x < 0$, the condition in the first branch is true, so

$$\text{Piecewise}((0, x < 0), (1, \text{True})) = 0.$$

Therefore, for every sufficiently small negative x , the expression is identically 0. The one-sided limit from the left is consequently

$$\lim_{x \rightarrow 0^-} p(x) = 0.$$

The returned value 1 is not an equivalent representation or a harmless branch choice. It is the right/default branch value: for instance

$$p(-1/10) = 0, \quad p(1/10) = 1.$$

A left-hand limit at this jump must be determined by the left-neighborhood values, not by substituting the boundary point into the branch condition.

1.5 Source-Level Diagnosis

The diagnosis identifies the source file `sympy/functions/elementary/piecewise.py`. The relevant method and traced call path are:

```
limit(p, x, 0, dir="-")
Limit(...).doit(deep=False)
Piecewise._eval_as_leading_term(x, logx, cdir)
```

The surrounding limit code in `sympy/series/limits.py` constructs a `Limit` object and evaluates it via `doit`. For a finite directed limit, the limit engine shifts the point of approach to 0 and records the left-hand direction as `cdir = -1`. It then asks the `Piecewise` expression for its leading term.

The relevant method receives the direction parameter but ignores it:

```
def _eval_as_leading_term(self, x, logx, cdir):
    for e, c in self.args:
        if c == True or c.subs(x, 0) == True:
            return e.as_leading_term(x)
```

For the condition $x < 0$, direct substitution at the boundary gives $(x < 0)|_{x=0}$, which is false. The method therefore skips the left branch even though `cdir = -1` means the approach is through negative values of the shifted variable. It then reaches the default `True` branch and returns the leading term of the expression 1, which the limit engine reports as the left-hand limit.

A plausible fix direction supported by the diagnosis is to make this `Piecewise` leading-term method direction-aware. For relational conditions involving the limit variable, it should test which branch holds in a punctured neighborhood determined by `cdir`, for example through a signed dummy substitution or an interval/set containment check, rather than by substituting the exact boundary point.

1.6 Additional Failing Instances

The independent verification checks the representative case together with five more constant jumps of the same form:

$$p_{a,b}(x) = \text{Piecewise}((a, x < 0), (b, \text{True})).$$

For each pair (a, b) , every sufficiently small negative x satisfies $x < 0$, so the expected left-hand limit is a . SymPy instead returns b , the default branch value.

Input / Expression	SymPy behavior	Expected behavior
$p_{0,1}$	returns 1, the default branch	returns 0, the left branch
$p_{2,5}$	returns 5, the default branch	returns 2, the left branch
$p_{-3,7}$	returns 7, the default branch	returns -3, the left branch
$p_{4,-2}$	returns -2, the default branch	returns 4, the left branch
$p_{11,13}$	returns 13, the default branch	returns 11, the left branch
$p_{-8,-1}$	returns -1, the default branch	returns -8, the left branch

The expected behavior is the same across the family because the condition $x < 0$ is true throughout a punctured left neighborhood of the limit point. The right/default branch may determine values at $x = 0$ or for $x > 0$, but it does not determine the left-hand limit.

1.7 Related Issues Assessment

The bug-finding harness initially classified this as a new failure mode with no public precedent, but that classification was incorrect. This is a known, long-standing problem in SymPy. It is the same root cause described in the open issue [#23836](#) (“Incorrect results for limits of Piecewise at discontinuity”), which states that the value of the `Piecewise` at the boundary should not be considered by `limit`, and it is a sibling of the open issue [#27236](#) (“bug with `limit()` function”), where one-sided limits of a discontinuous `Piecewise` return the wrong branch. There are also open pull requests targeting the precise function diagnosed above (`Piecewise._eval_as_leading_term` ignoring the direction `cdir`): [#29589](#), [#29538](#), and [#29920](#). The closest related public material is therefore not merely subsystem-level: these issues and PRs describe this exact failure. The bug remains a clear, reproducible instance of this known family, and the focused regression test below guards branch selection in directed limits of `Piecewise` expressions.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import Piecewise, limit, symbols

@pytest.mark.parametrize(
    "left_value, default_value",
    [(0, 1), (2, 5), (-3, 7), (4, -2), (11, 13), (-8, -1)],
)
def test_piecewise_left_hand_limit_uses_left_branch(left_value, default_value):
    x = symbols("x", real=True)
    p = Piecewise((left_value, x < 0), (default_value, True))
    assert limit(p, x, 0, dir="-") == left_value
```